

Transformer (Decoder)

Natural Language Processing

Ayesha Enayet

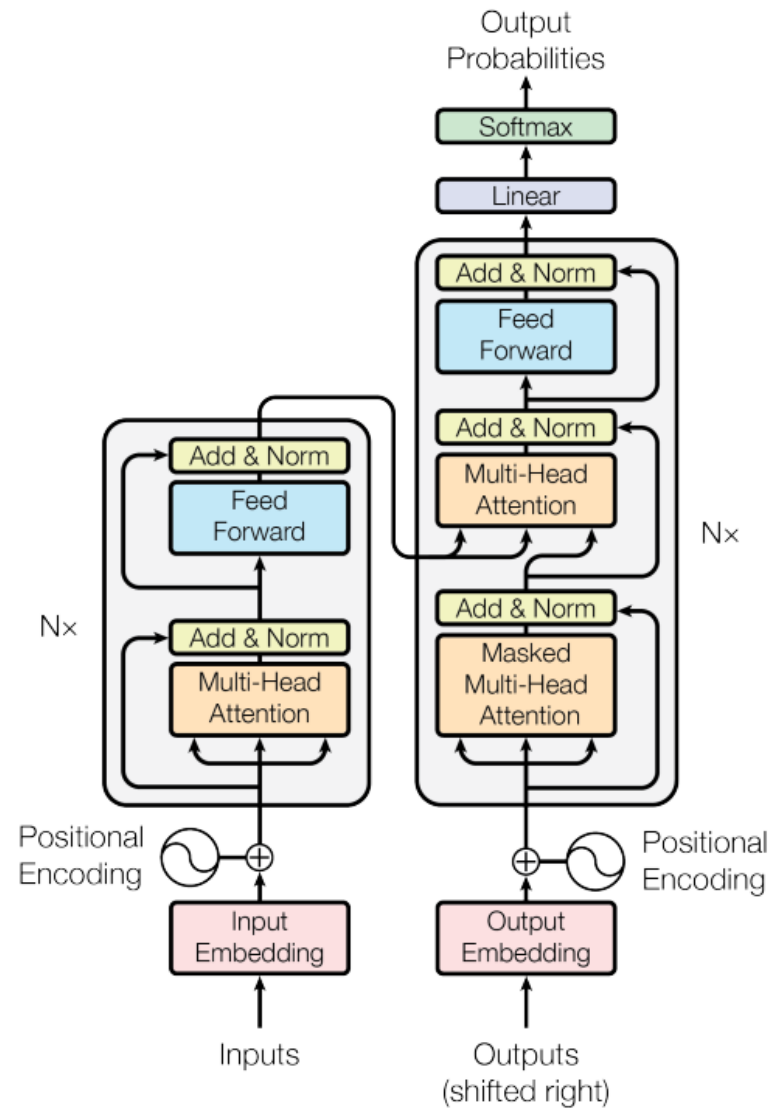
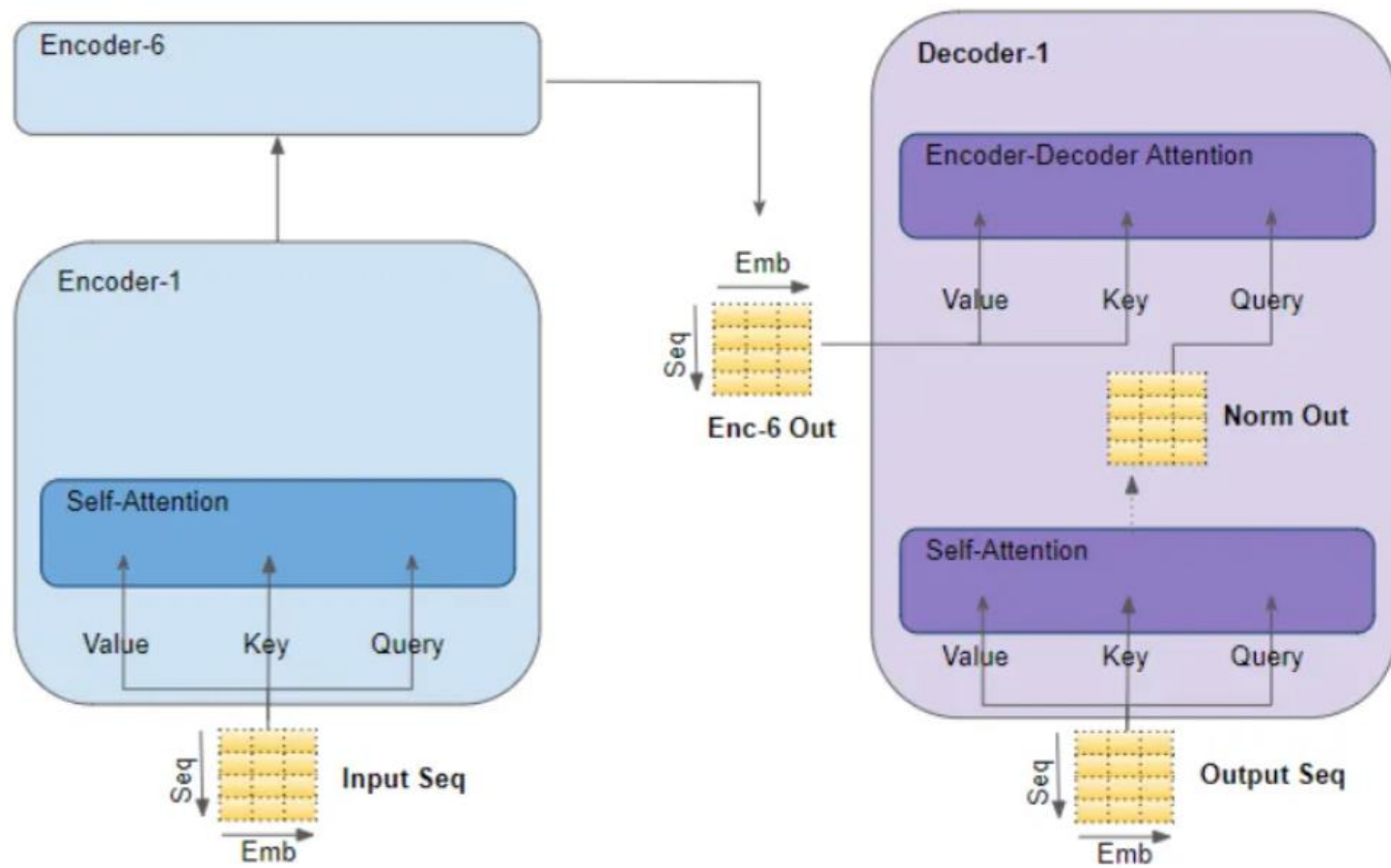


Figure 1: The Transformer - model architecture.



<https://medium.com/@YanAlx/step-by-step-into-transformer-79531eb2bb84>

Attention

Apply Mask here after
finding the dot product

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Masked Multi-head Attention

- The masked multi-head attention in the decoder ensures sequential processing of tokens, a method that prevents each generated token from being influenced by subsequent tokens. This masking is important for maintaining the order and coherence of the generated data.
- We apply masking after finding a dot product between Q and K and before softmax.
 - For first iteration we just add 0 to the output of the dot product.

Example (Masked attention matrix)

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

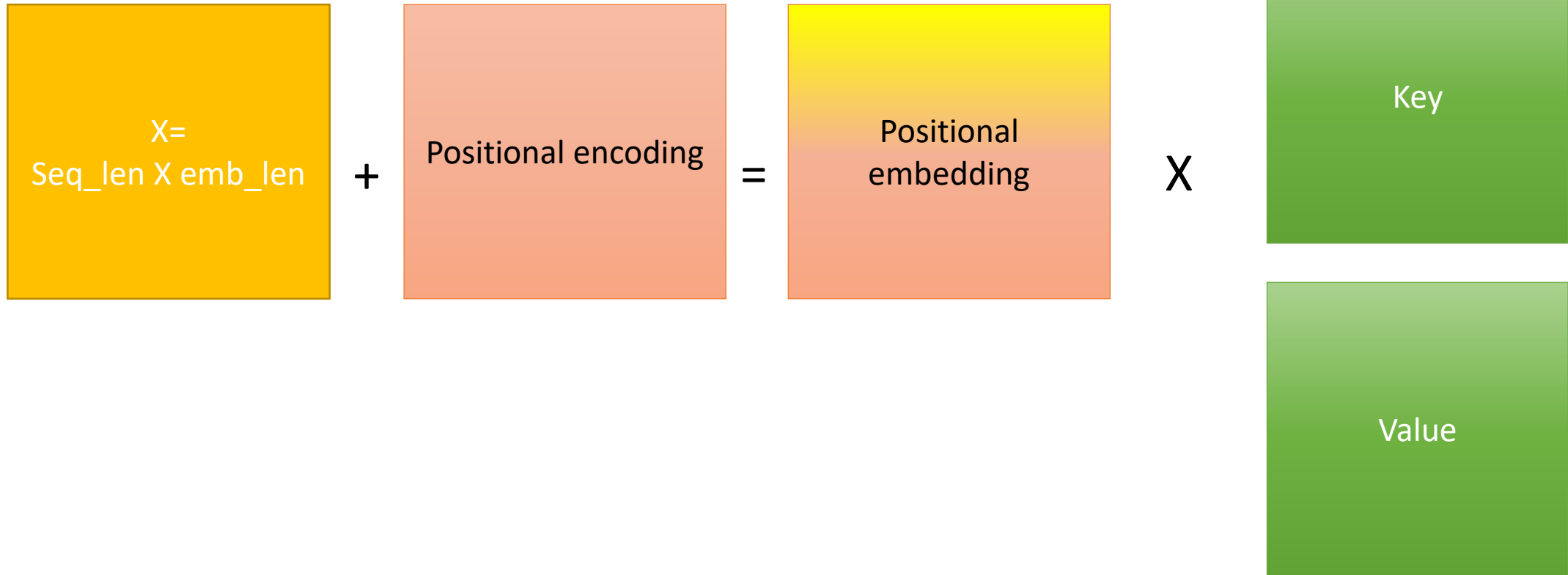
$$\begin{bmatrix} 0.71 & 0.01 & 0.6 \\ 0.77 & 0.34 & 0.34 \\ 0.45 & 0.56 & 0.32 \end{bmatrix} + \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.71 & -\infty & -\infty \\ 0.77 & 0.34 & -\infty \\ 0.45 & 0.56 & 0.32 \end{bmatrix}$$

$$\frac{QK^T}{\sqrt{d_k}} + M$$

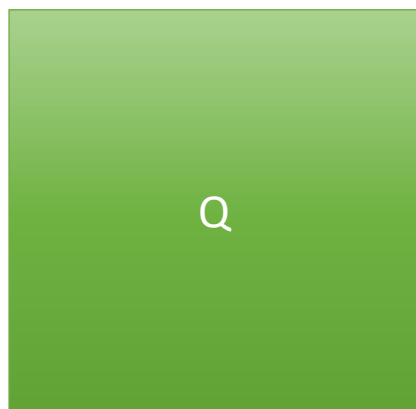
Dynamic initialization of Mask

- ```
if Attention_scores.ndim > 1:
 M = np.zeros(Attention_scores.shape)
 for i in range(Attention_scores.shape[0]):
 for j in range(i+1, Attention_scores.shape[1]):
 M[i,j] = -np.inf
else:
 M = 0
```

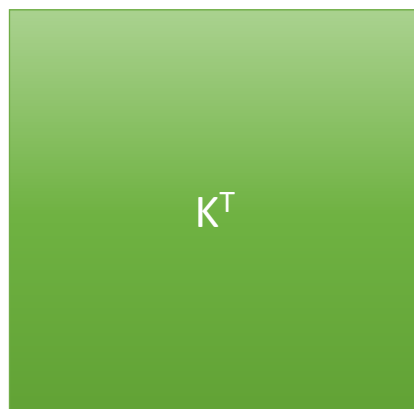
# Decoder Training



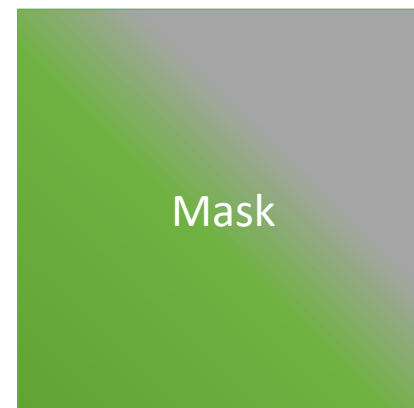




$\times$

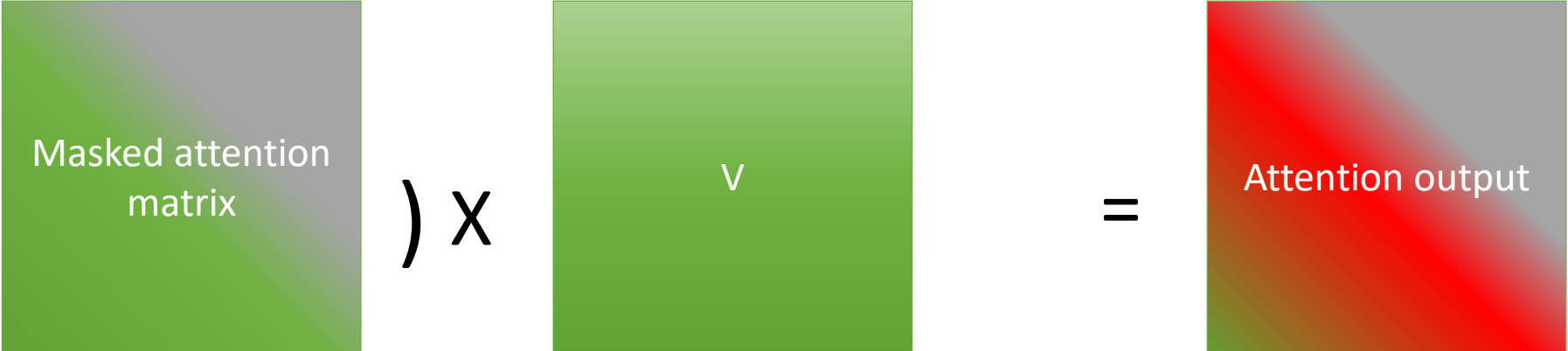


+



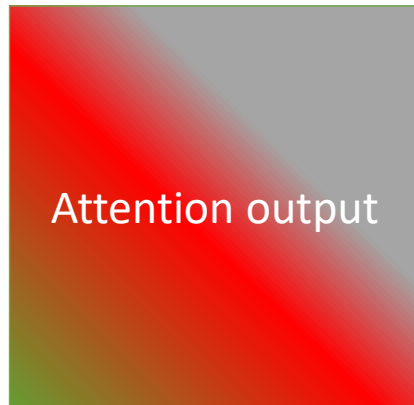
---

$\sqrt{dk}$

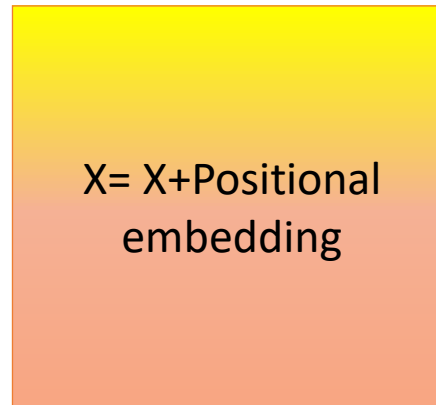
$$\text{Softmax}(\text{Masked attention matrix}) \times V = \text{Attention output}$$


Note: incase of multi-head attention you will replicate a process  $h$  times and concatenate the out and transform the dimension to make it equal to the input dimension.

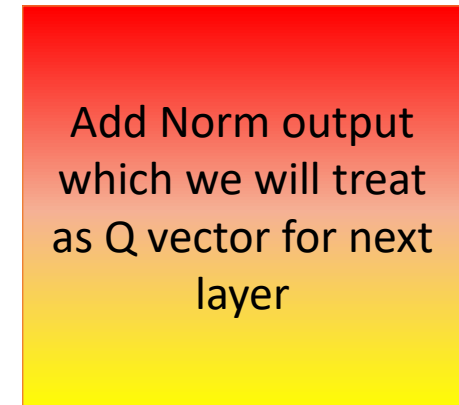
## Add Norm

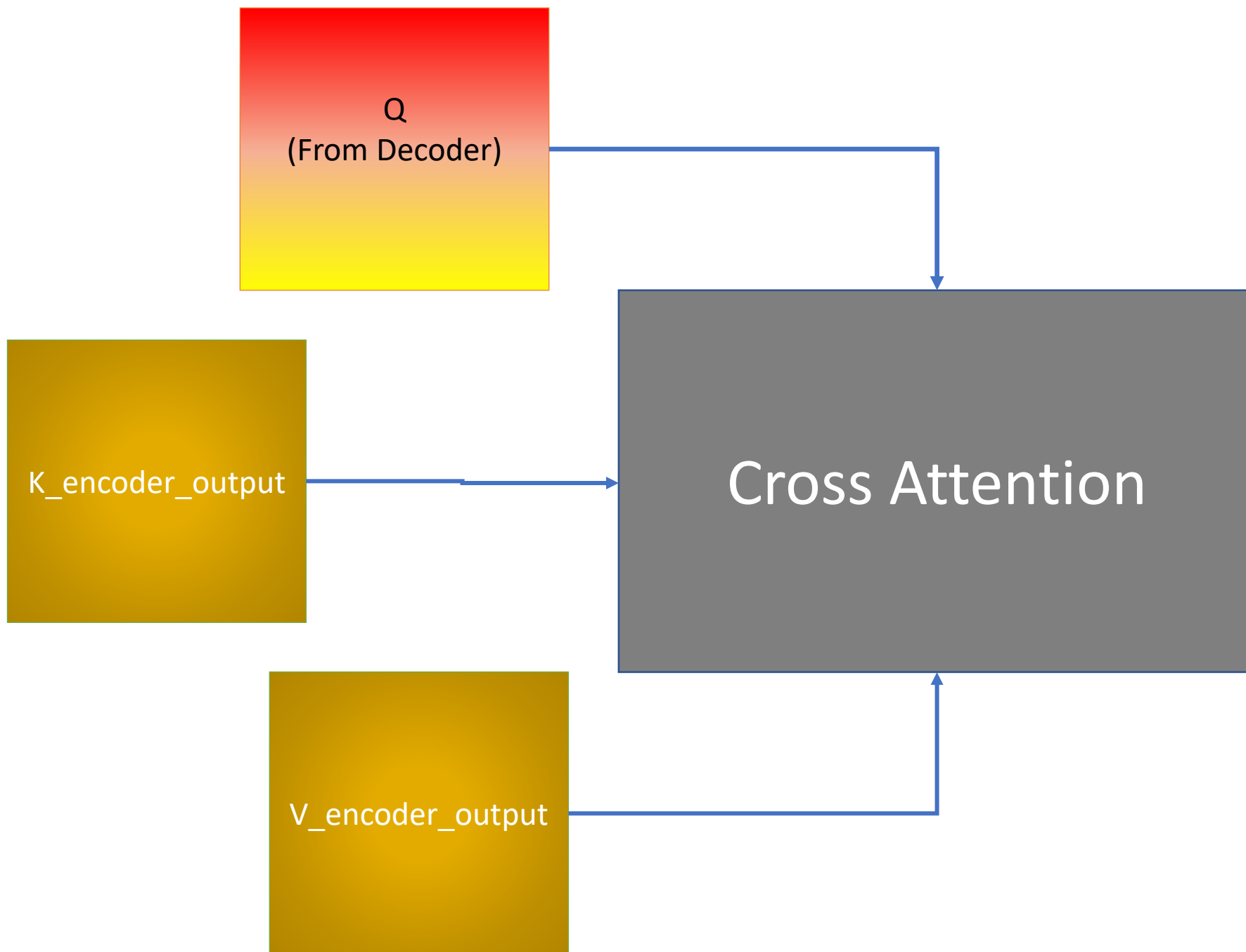


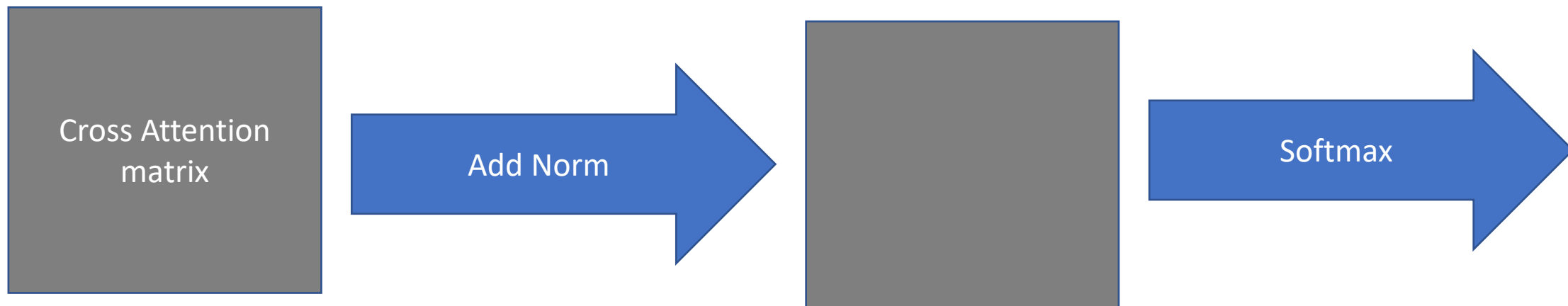
+



=









# Decoder input (Testing)

- First iteration : Input embedding for <Start> token
- Add positional embedding
- Map to three matrices Q, K, V
- Dimensions: 1 X second\_dimension\_QKV

# Decoder output

- A predicted token
- We will append the predicted token after the last input and pass it through the decoder for next prediction.



# Credit: Detailed Explanation

- [Blowing up Transformer Decoder architecture](#)
- <https://zaai.ai/lms-and-transformers-from-scratch-the-decoder/>

# Foundation Models

# Some Famous FMs

- BERT
  - Encoder-only model
  - Classification tasks
- GPT
  - Decoder-only model
  - Sequence-to-sequence tasks or language generation
- LLaMa
  - Decoder-only
  - Sequence-to-sequence tasks or language generation
- T5 & BART
  - Encoder-Decoder model